Topic:

How do different pooling strategies in convolutional neural network architectures (max, min, and average pooling) affect clothing recognition accuracy and feature extraction?

Subject Area:

Computer Science

Word Count: 3350

Table of Contents

---

1 Introduction 3

1.1 Layers 4

1.2 Pooling 5

1.3 Object Recognition 6

2 Background 6

2.1 Max Pooling 6

2.1.1 Formula 7

2.1.2 Most useful 7

2.2 Min Pooling 8

2.2.1 Formula 8

2.2.2 Most useful 9

2.3 Avg Pooling 9

2.3.1 Formula 10

2.3.2 Most useful 10

3 Methodology 10

3.1 Dataset 10

3.2 Evaluation Metrics 11

3.3 Hypothesis 12

4 Results and Analysis 13

4.1 Tabular Representation 13

4.1.1 Precision 14

4.1.2 Recall 15

4.1.3 F1-Score 15

4.2 Graphical Representation 15

4.2.1 F1-Confidence 15

4.2.2 Precision-Recall 19

4.2.3 Precision & Recall on Confidence 22

4.2.5 Confusion Matrix 25

5 Conclusion 27

6 Limitations and Challenges 28

7 Bibliography 29

8. Appendices 31

List of Figures

Figure 1: CNN compared to a brain 4

Figure 2: CNN layers 5

Figure 3: Max Pooling 6

Figure 4: Min Pooling 8

Figure 5: Avg Pooling 9

Figure 6: Fashion MNIST dataset images 11

Figure 7: Basic Confusion matrix representation 12

Figure 8: Results of Training and Testing the Pooling Strategies 13

Figure 9: Results of Precision, Recall, and F1-Score tests of the Pooling Strategies 14

Figure 10A, 10B, 10C: Graphs of F1-Score vs Confidence Threshold tests of the Max, Min, Avg Poolings respectively 18

Figure 11A, 11B, 11C: Graphs of Precision vs Recall tests of the Max, Min, Avg Poolings respectively 21

Figure 12A, 12B, 12C: Graphs of Precision & Recall vs Confidence Threshold tests of the Max, Min, Avg Poolings respectively 24

Figure 13A, 13B, 13C: Graphs of the Confusion Matrixes of the Max, Min, Avg Poolings respectively 27

INTRODUCTION

Over the past years, in this expeditious world, the development of Artificial Neural Network (ANN) has improved unbelievably far. In the 2020s, an ongoing period of rapid progress in advanced generative AI became known as the AI boom. Among the core technologies of this boom are Artificial Neural Networks, which represent the human brain in processing and learning from data. Artificial Neural Networks are made of different architectures, each designed to work on specific tasks. For this investigation, to explore different pooling strategies (max, min, and average pooling) that affect clothing recognition accuracy and feature extraction, Convolutional Neural Network (CNN) will be used, as it is a specialized type of deep learning algorithm mainly designed for tasks that necessitate object recognition, including image classification, detection, and segmentation.

Convolutional neural networks were inspired by the layered structure of the human visual cortex. Like the brain, CNNs first detect simple features and then combine them in deeper layers to recognize more complex patterns.

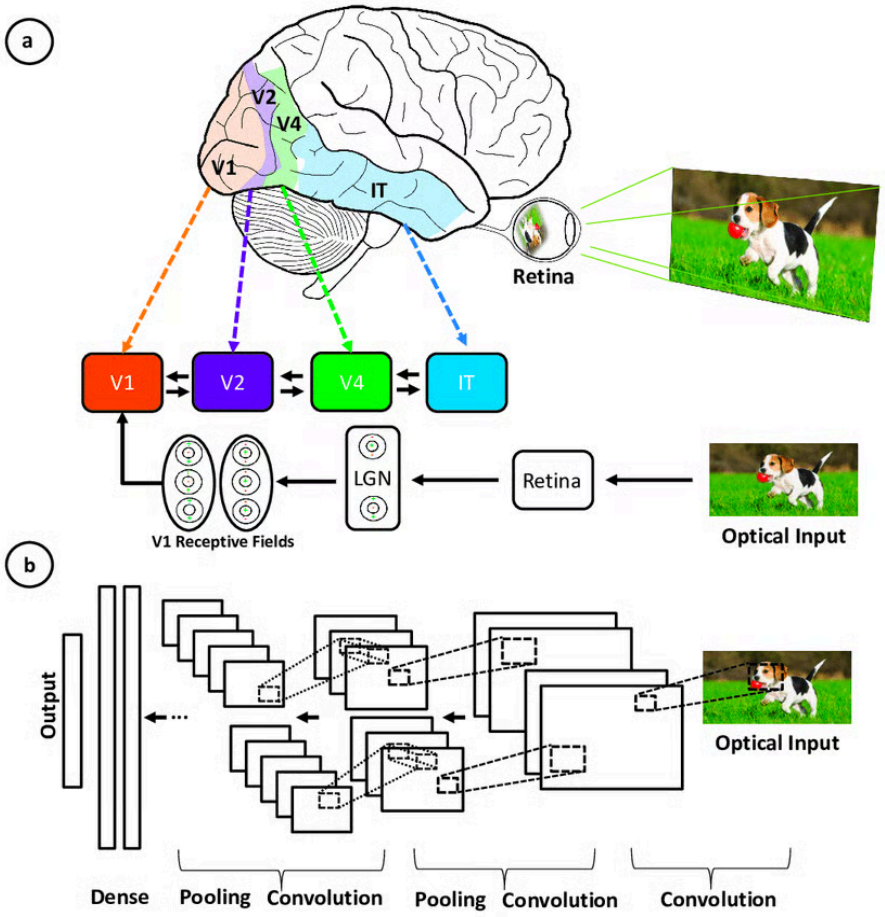


Fig. 1 - CNN compared to a brain

(Nisha Bhadauriya Agarwal - 2024)

Although simpler than the visual cortex, CNNs capture the main ideas of human vision, making them suitable for tasks like object recognition and studying different pooling strategies.

1.1 Layers

Inside, ANNs are composed of interconnected artificial neurons, each of which receives inputs, applies weights and a bias, and generates an output through an activation function. These neurons are organized into distinct layers (Fig. 2).

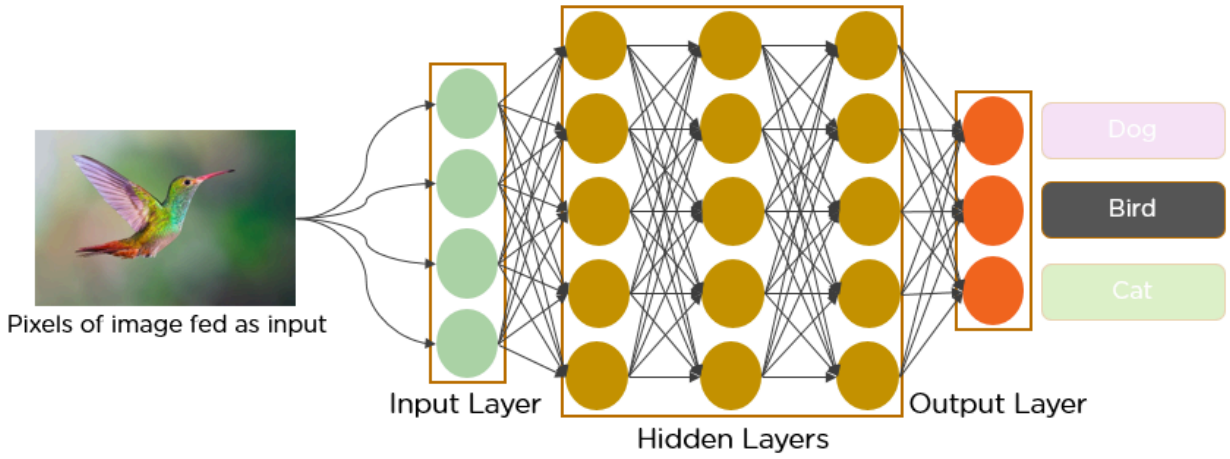


Fig. 2 - CNN layers

(Pranshu Sharma - 2024)

The input layer is responsible for receiving user's data, such as images or text, and sending it into the network. *The hidden layers*, which may consist of several layers, transform the input data by extracting and refining it through activation functions. Finally, *the output layer* produces the final result, such as classifying an object in an image or recognizing a pattern in text.

1.2 Pooling

Hidden layers commonly employ three pooling strategies: max pooling, min pooling and average pooling. The function of the pooling layers is to reduce the size or the spatial dimensions (width & height) of the image. “Max pooling selects the largest values, which can result in the loss of relevant information in deeper layers and restricts gradient flow.” (See “Comprehensive Evaluation …” (Source 1) and “How to Backpropagate for CNNs” (Source 2).) In min pooling the summary of the features in a region is represented by the minimum value in that region. It is mostly used when the image has a light background since min pooling will select darker pixels. Average pooling assigns equal weight to all inputs, which can diminish the influence of individual neurons.

1.3 Object Recognition

In the first layers, the network tries to detect simple elements such as lines, edges, and curves. Then they are combined in deeper layers to form more complex figures, such as polygons representing the shape of the clothing or the outlines of its parts. Pooling layers then condense these details, keeping the most relevant features. Through this hierarchical process, the CNN constructs feature maps that allow it to identify a type of clothing, even under varying lighting or orientation. “A feature map in CNN is the output representation obtained after applying convolutional filters to the input data. It highlights patterns and structures such as edges, textures, or object parts, depending on the layer’s depth in the network.” (Source 3)

BACKGROUND:

2.1 Max Pooling

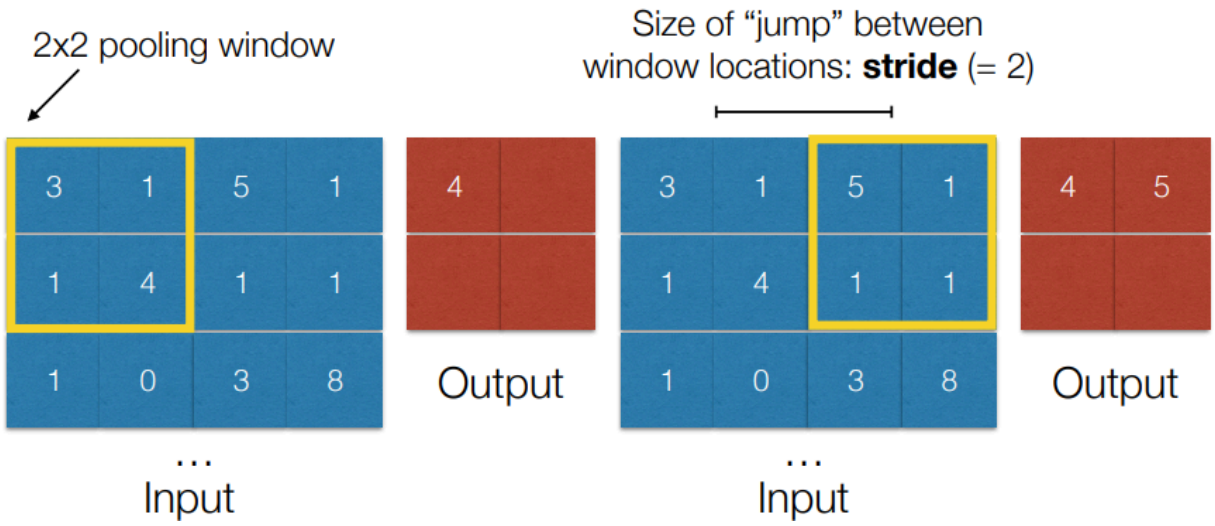


Fig. 3 - Max Pooling

(Colby College - 2022)

Max pooling is a pooling method that selects the maximum element from the region of the feature map covered by the filter. Max pooling has a couple characteristics: pooling window – kernel, or size of "jump" – stride. Kernel in pooling strategies is a small, fixed-size window that slides across the input feature map, it is usually 2x2 (like in the photo) or 3x3. Stride defines the number of pixels by which the pooling kernel moves across the input feature map after each operation. For example a stride of 2 means that the kernel moves 2 pixels at a time, causing the output to become smaller. Bigger stride means smaller output.

2.1.1 Formula:

Let's say:

- Input Feature Map = X
- Pooling window (kernel size) = $k \times k$
- Stride = s
- i, j tell us which window we are considering (where we are in the output map).
- m, n iterate over all pixels inside the window.

Then the max pooling output at location (i, j) is:

$$Y_{i,j} = max\, X_{(i \times s+m),(j \times s+n)}$$

Where $0 < m < k$ and $0 < n < k$.

2.1.2 Most useful:

- Noise reduction
    - Noise usually produces small random activations which are ignored because only the strongest survives.
- Highlighting the contours

- If an edge moves slightly in an image, max pooling still captures it as long as it's within the pooling window.

- Feature Detection
  - Important features like edges and corners are usually highlighted by high activations, and max pooling ensures they are not "averaged out."

2.2 Min Pooling

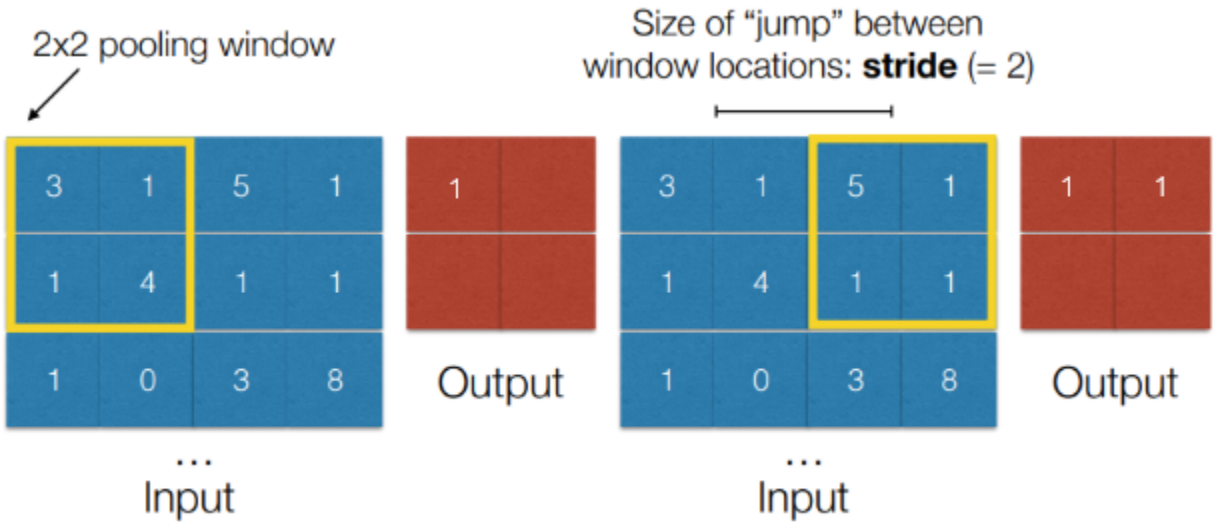


Fig. 4 - Min Pooling

(Colby College -2022, Edited by Myself)

Min pooling is a pooling strategy in CNNs where we take the minimum value inside the window. This approach highlights darker or less intense regions of an image, which can sometimes correspond to shadows, contours, or background elements. Min pooling also uses characteristics like stride and kernel to help to find the best way of achieving the goal.

2.2.1 Formula:

$$Y_{i,j} = min\, X_{(i\times s+m),(j\times s+n)}$$

Where $0 < m < k$ and $0 < n < k$.

2.2.2 Most useful:

- Background extraction
    - Min pooling can emphasize background pixels rather than strong features because it captures the least activated regions, in other words, the parts of the image where the convolutional filters did not work well due to the absence of a pattern.
- Noise filtering
    - If an image has bright noise, min pooling can suppress it by focusing on darker values.

2.3 Average Pooling

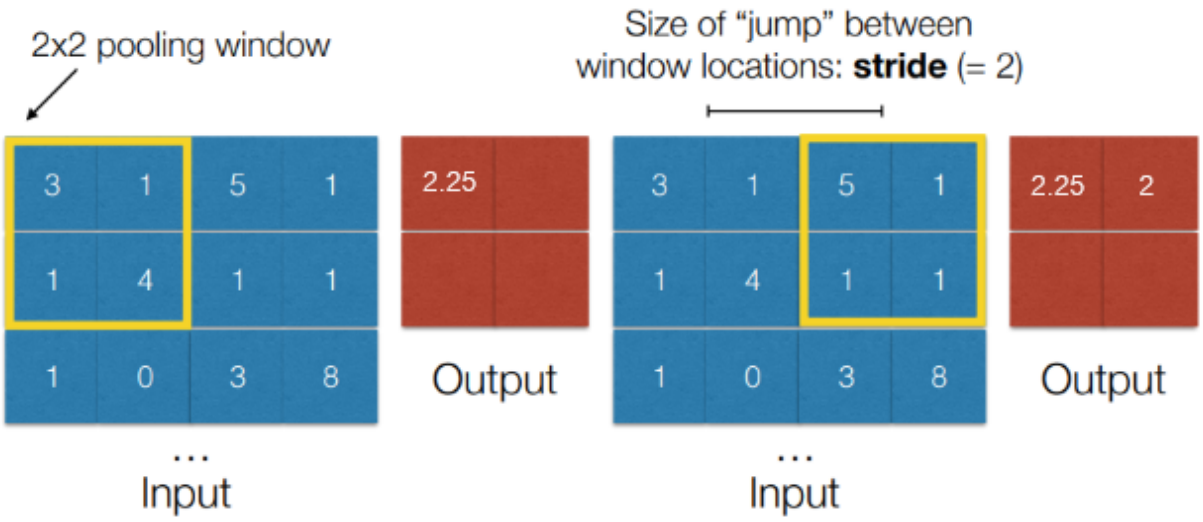


Fig. 5 - Avg Pooling

(Colby College - 2022, Edited by Myself)

Average Pooling is an operation in CNNs where we take the average of all values in a window. Unlike max or min poolings, it preserves the overall intensity or trend in a region rather than highlighting the strongest or weakest spot.

2.3.1 Formula

$$Y_{i,j} = \frac{1}{k^2} \sum_{m=0}^{k-1} \times \sum_{n=0}^{k-1} \times X_{(i \times s+m),(j \times s+n}$$

Where $0 < m < k$ and $0 < n < k$.

2.3.2 Most useful

- Smoothing & Noise Reduction
    - If your input images have random high-intensity noise, average pooling reduces its impact by considering the mean value in the window.
- Texture Recognition
    - In textures or patterns that are distributed rather than localized, max pooling may discard subtle but important details.

METHODOLOGY

3.1 Dataset

Fashion MNIST is an online directory that consists of 60.000 grayscaled 28×28 images for training and 10.000 images with the same characteristics for testing. Each image is referred to 1 of 10 clothing classes. The dataset can be accessed through an Application Programming Interface (API) using Google Colaboratory, a cloud-based platform that allows users to run Python code online without installing local software. This platform was therefore used for this investigation to train and test the model (see Appendix A).

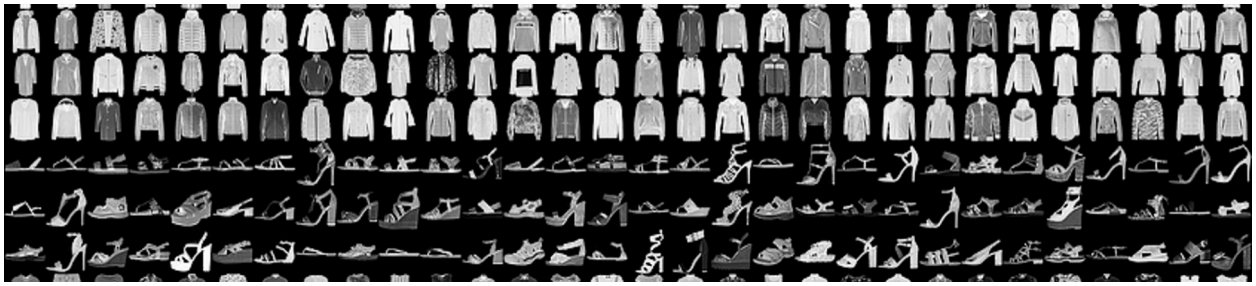


Fig. 6 - Fashion MNIST dataset images

(Zalando - 2017)

3.2 Evaluation Metrics

Confusion matrix, F1-Confidence, Precision-Recall, Precision-Confidence, and Recall-Confidence were used as evaluation criteria for the results of each pooling strategy. These evaluation metrics will be explained in detail later in this essay.

Precision tells how often the model's positive predictions are correct, while recall measures how often the model successfully identifies positive cases among all actual positives. The F1 score combines the precision and recall scores of a model. It shows how well the model's balance is between precision and recall.

The confusion matrix is a table used to measure how well a model is performing, it tells us where it gets confused. Confusion matrix uses classes for binary classification, those are:

- True Positive (TP): Instances are correctly predicted as they belong to the positive class.
- False Positive (FP): Instances are incorrectly predicted as they belong to the positive class when they actually belong to the negative class.

- True Negative (TN): Instances are correctly predicted as they belong to the negative class.
- False Negative (FN): Instances are incorrectly predicted as they belong to the negative class when they actually belong to the positive class.

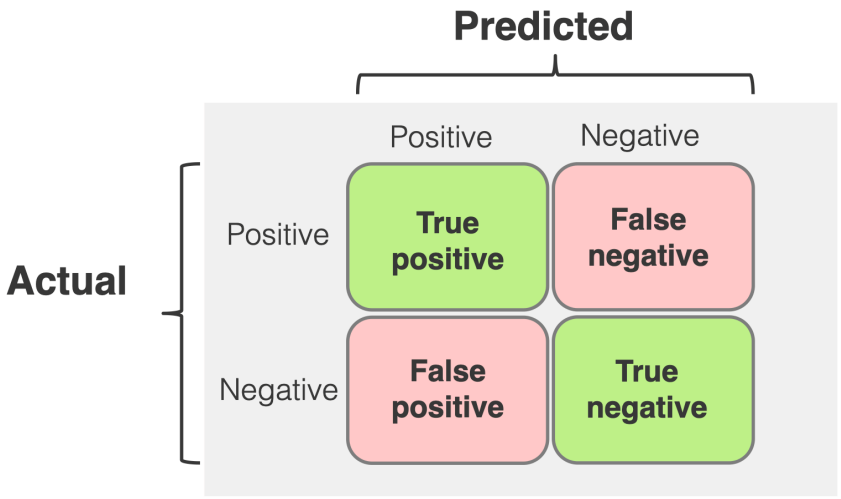


Fig. 7 - Basic Confusion matrix representation

(Rishabh Singh - 2024)

3.3 Hypothesis

Every pooling method has its own advantages and disadvantages in object recognition, however, for clothing recognition we need a method that is able to highlight the contours and edges of different instances. The max pooling is the best fit for this task, as it specifically works with feature detection assignments and therefore will show the best results. The average pooling, on the other hand, might discard important details and miss the corners. And the min pooling would be a good decision if the clothing was black and the background was white (vice-versa to Fashion-MNIST colors)

RESULTS AND ANALYSIS

4.1 Tabular Analysis

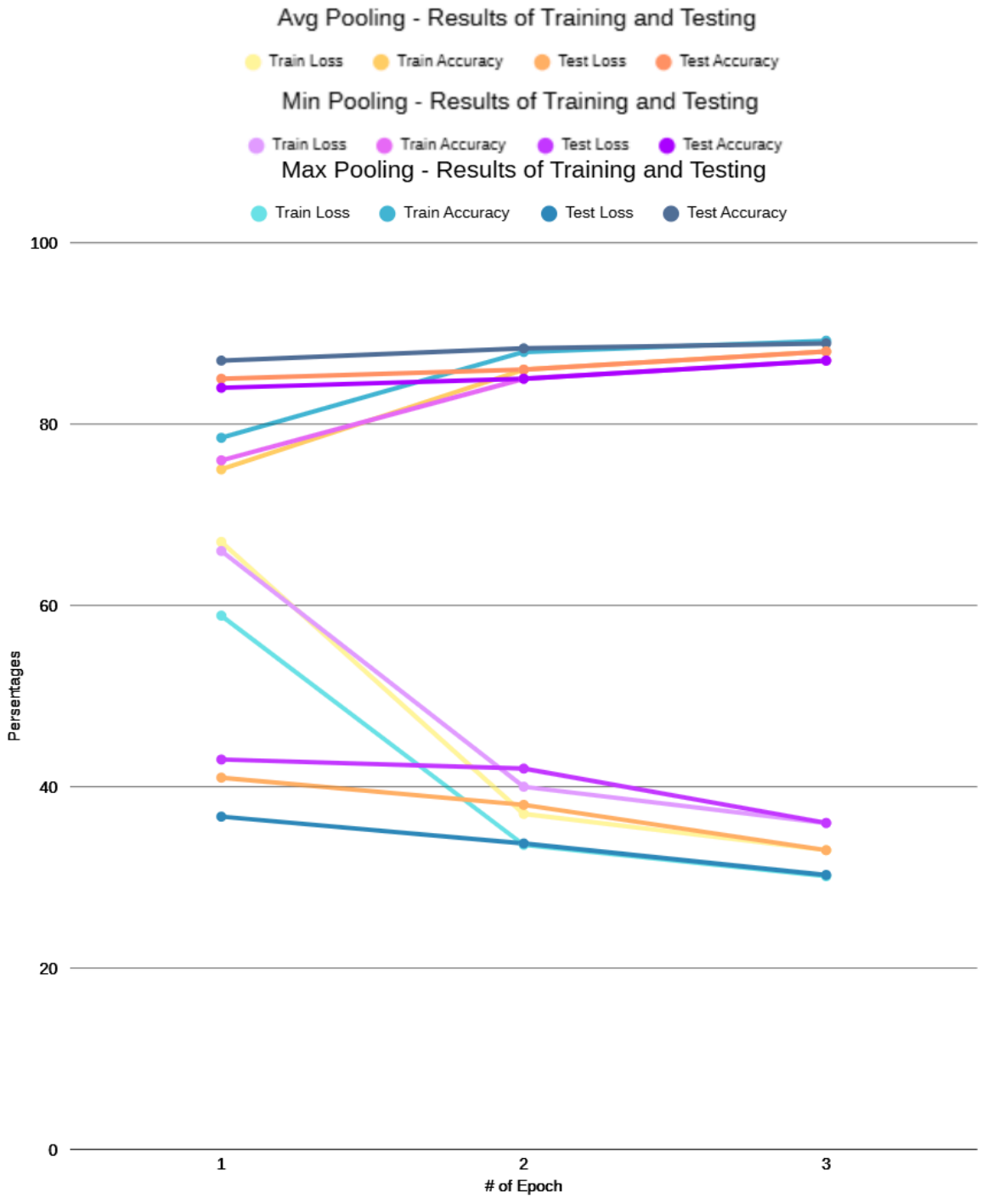


Fig. 8 - Results of Training and Testing the Pooling Strategies

(Made by Myself - 2025)

After 3 epochs, each model showed a clear presence of model improvement. We can see that the training loss consistently becomes lower and lower, especially after the first epoch, indicating that the model quickly adapted to the data. Train accuracy significantly improves at the same time.

Testing has similar trends. Both the test loss and test accuracy showed an improvement in data recognition: test loss decreased, and test accuracy showed an increase.

	Max Pooling			Average Pooling			Min Pooling		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
T-shirt/top	0.89	0.77	0.82	0.86	0.79	0.82	0.85	0.79	0.82
Trousers	0.97	0.98	0.98	0.97	0.97	0.97	0.96	0.97	0.97
Pullover	0.84	0.83	0.84	0.81	0.81	0.81	0.79	0.79	0.79
Dress	0.86	0.89	0.87	0.87	0.87	0.87	0.87	0.86	0.87
Coat	0.77	0.87	0.82	0.75	0.81	0.78	0.74	0.83	0.78
Sandal	0.98	0.98	0.98	0.98	0.96	0.97	0.98	0.95	0.97
Shirt	0.71	0.66	0.68	0.66	0.65	0.65	0.66	0.6	0.63
Sneaker	0.95	0.97	0.96	0.93	0.97	0.95	0.93	0.96	0.94
Bag	0.96	0.98	0.97	0.97	0.97	0.97	0.96	0.96	0.96
Ankle boot	0.98	0.96	0.97	0.97	0.95	0.96	0.95	0.95	0.95
Average:	0.8894	0.889	0.8882	0.8767	0.8761	0.876	0.8675	0.868	0.8672

Fig. 9 - Results of Precision, Recall, and F1-Score tests of the Pooling Strategies

(Made by Myself - 2025)

4.1.1 Precision

Max pooling shows the highest average precision (0.8894) compared to the other pooling strategies. That means that, out of all the positive predictions made by the model, 88.94% were correct (true positive), while only 11.06% were incorrect (false positives).

Min pooling provides the lowest average precision results (0.8675), which makes it do only 86.75% correct (true positive) results and 13.25% false positives.

Average pooling placed in the middle-group, showing the results of 0.8767. So, 87.67% answers were true positives, and 12.33% false positives.

4.1.2 Recall

Maximum pooling shows the highest average recall value (0.889), meaning that the model had 88.9% successful identifications of all actual positives, while failing to detect the remaining 11.1%.

Min pooling gave the lowest average recall score (0.868), representing the 86.9% successful true positive results. Even though this is the lowest result, it placed pretty close to the highest result, which tells us that all models have adapted really well.

Average pooling placed second, as the average recall score is 0.8761 or 87.61% of successful true positive results.

4.1.3 F1-Score

F1-Score combines both recall and precision, making it the most powerful way to value the data.

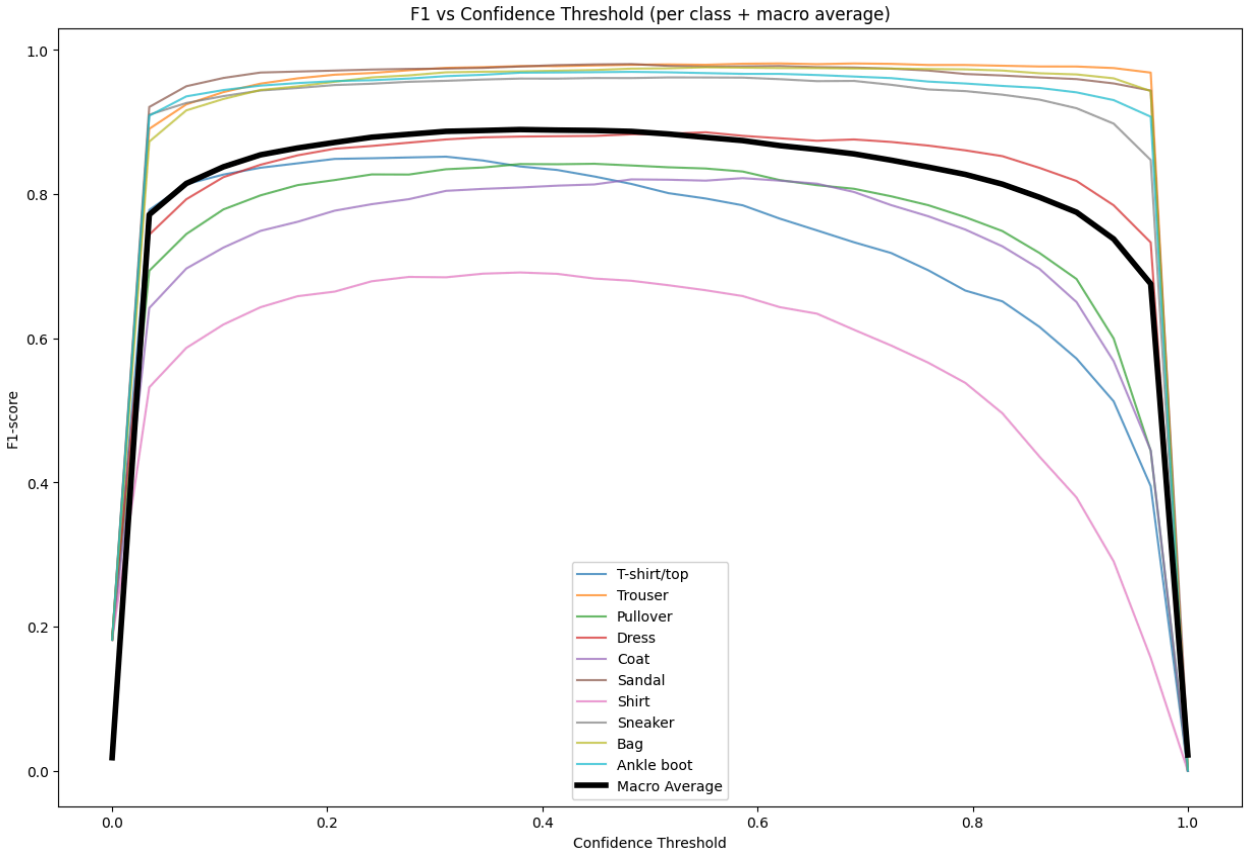
Max pooling has the highest F1-Score (0.882) along all 3 pooling models, which tells us that this pooling strategy is accurate (it makes relatively few false positive predictions and captures most of the true positive values).

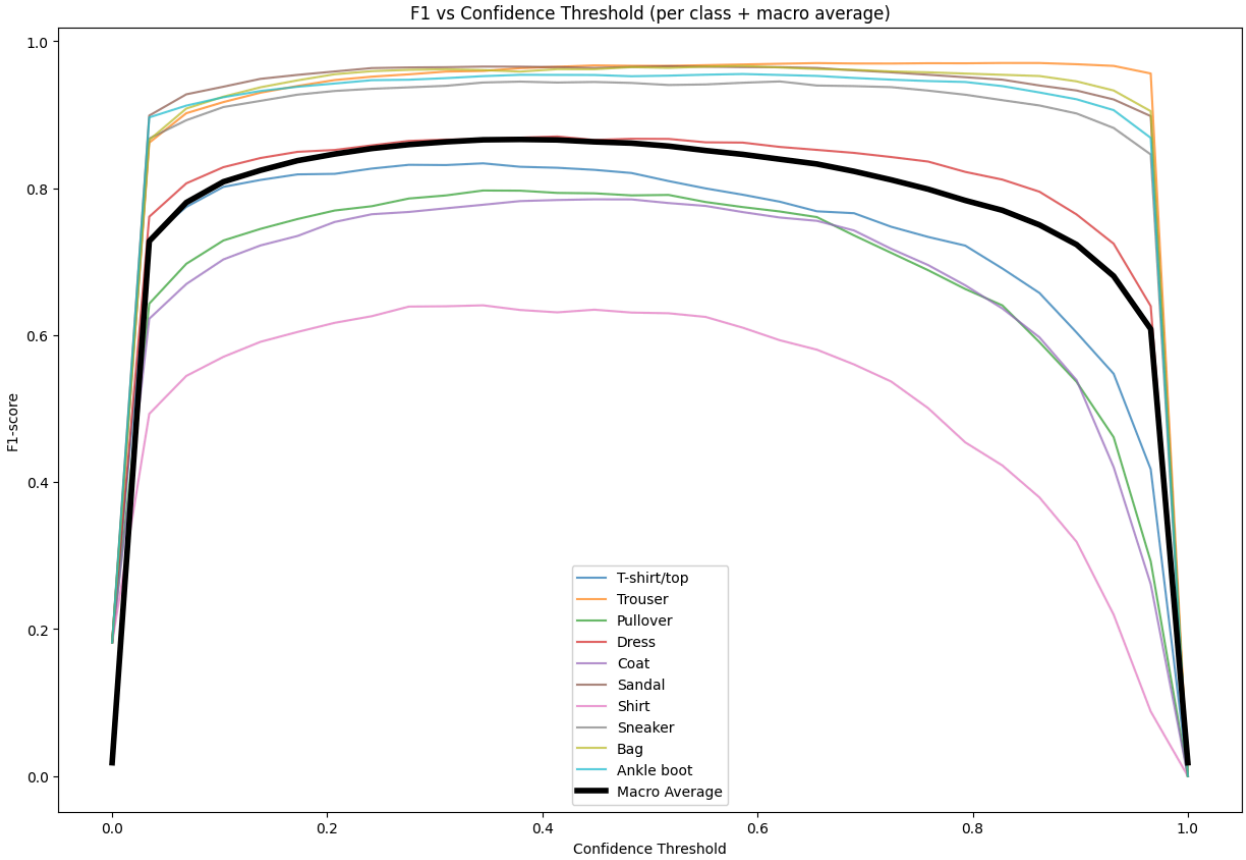
Min pooling has the lowest F1-Score (0.8672), which is still a great result, as it shows that neither precision nor recall is falling behind and therefore it is a well-balanced result.

Average pooling is in second place with the F1-Score of 0.876, demonstrating that it also shows a great balance between precision and recall, though not as effective as max pooling.

4.2 Graphical Analysis

4.2.1 F1-Confidence


F1 vs Confidence Threshold (per class + macro average)
F1-score
Confidence Threshold
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle boot
Macro Average


F1 vs Confidence Threshold (per class + macro average)
F1-score
Confidence Threshold
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle boot
Macro Average

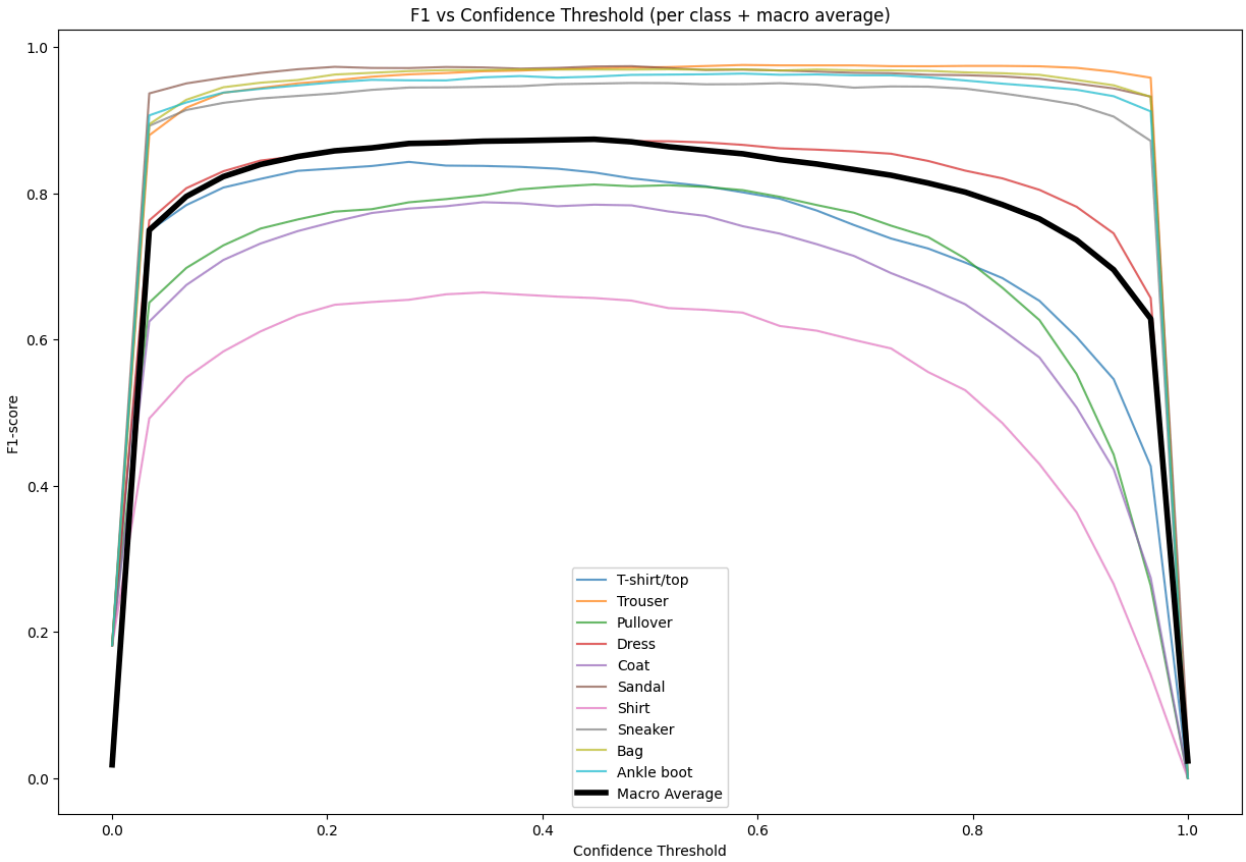


Fig. 10A, 10B, 10C - Graphs of F1-Score vs Confidence Threshold tests of the Max, Min, Avg Poolings respectively (Made by Myself - 2025)

The F1-score vs confidence threshold graphs show how the model's balance between precision and recall changes as we need the model to be more confident before predicting a class. Low threshold (0-0.3) shows how the model predicts many samples and therefore causes high recall and low precision. High threshold (0.7-1) shows how the model predicts only when it is very confident, causing high precision and low recall. The curve tells us how robust each pooling method is as we ask for stricter predictions.

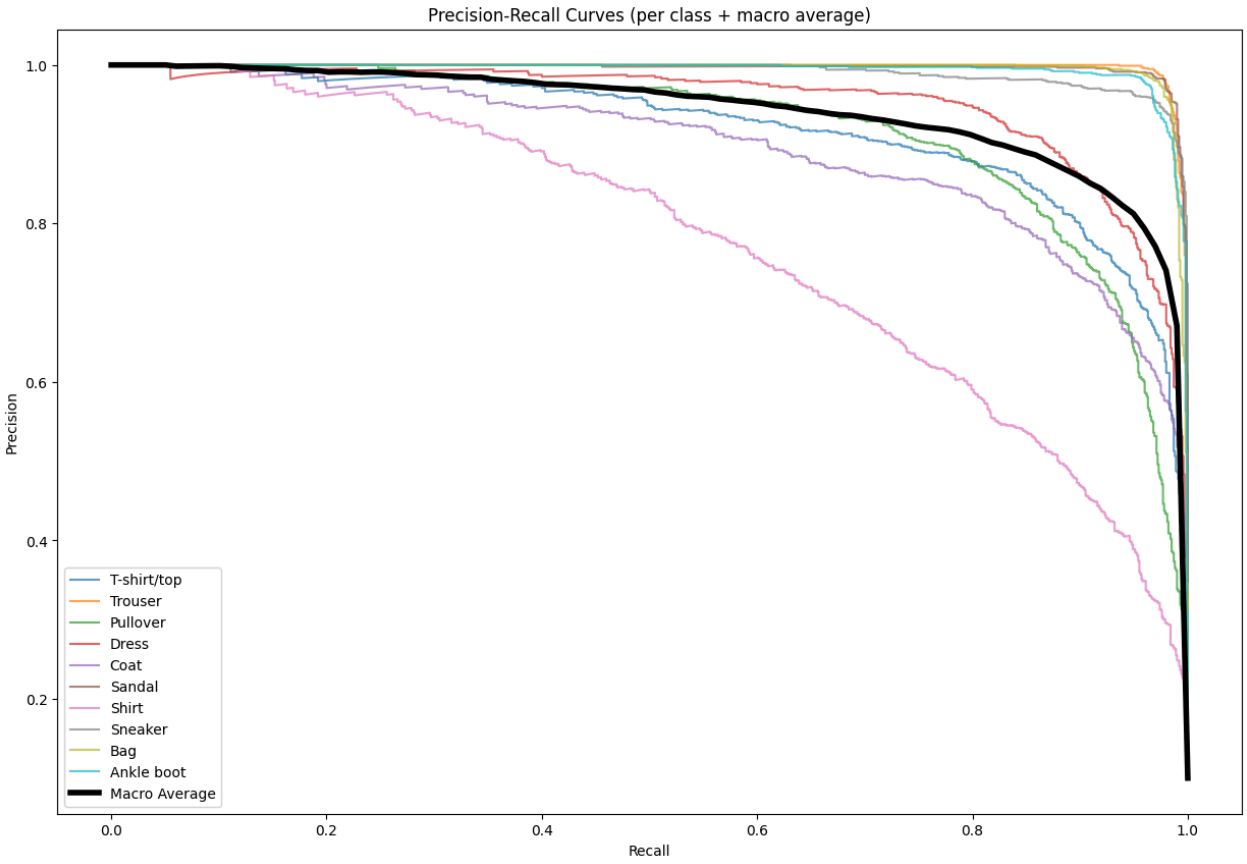
The 'F1-Confidence' graphs demonstrate the expected results, as when the threshold value is low (many predictions are positive), the models show high recall and low accuracy. F1,

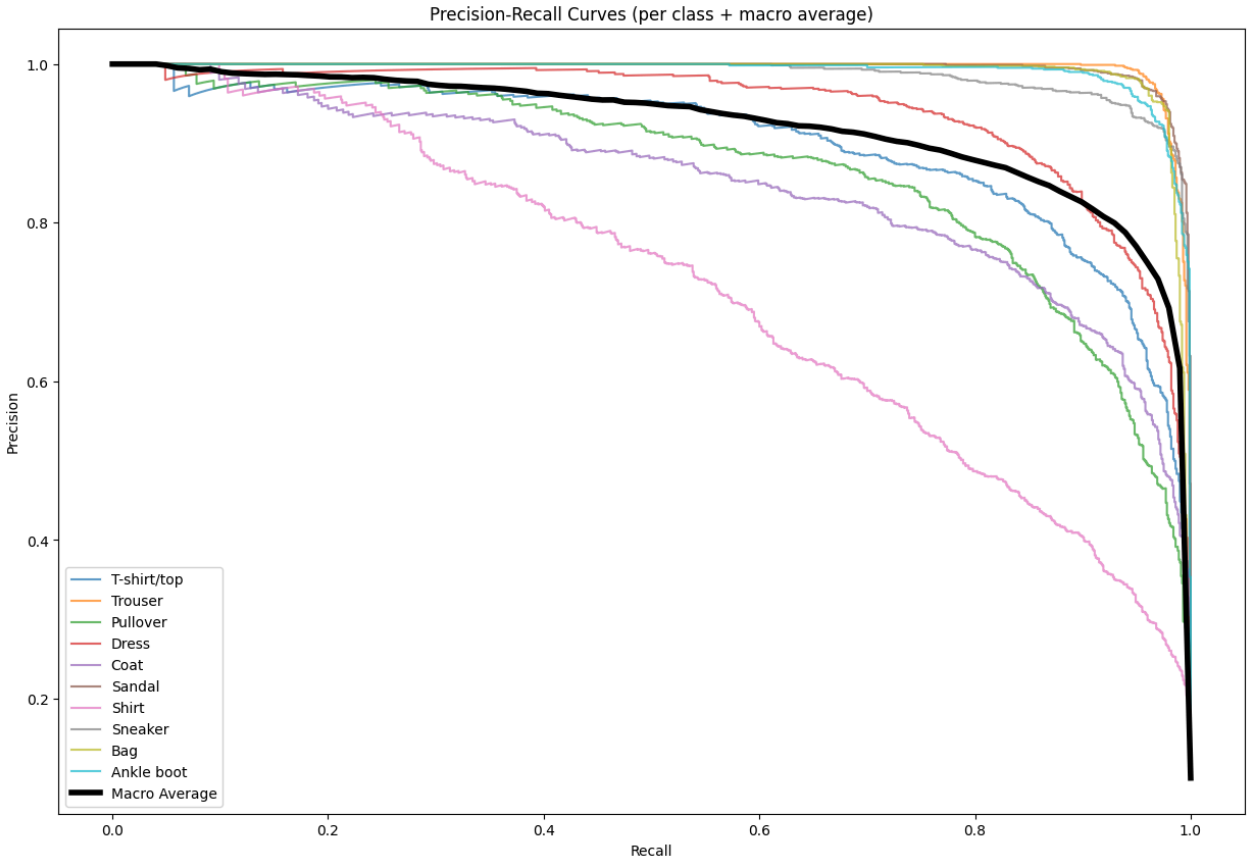
as it finds the mean between precision and recall, usually has a maximum point in some intermediate point, where both these factors are balanced.

If we take a closer look at a model with Max pooling, we can see that it shows better performance over Min and Avg poolings. This finding supports our tabular analysis and can be justified by the fact that max pooling works well with boundaries and the edges recognition.

According to data from Figures 10, trousers and sneakers showed the highest results, the possible reason is that trousers have a simple vertical shape, making them easy to detect for a model, and sneakers, on the other hand, have a complex outline that sets them apart from any other class.

4.2.2 Precision-Recall




Precision-Recall Curves (per class + macro average)
Precision
Recall
1.0
0.8
0.6
0.4
0.2
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle boot
Macro Average

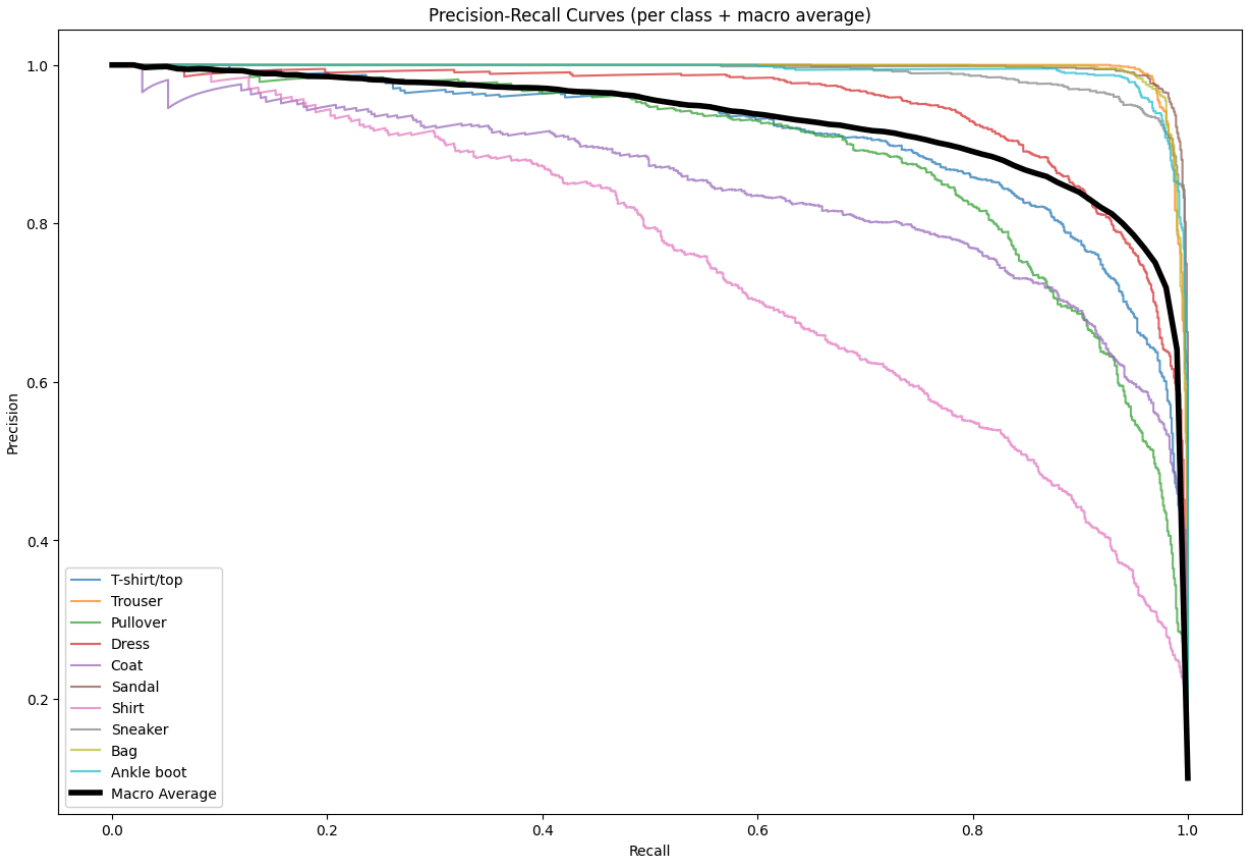


Fig. 11A, 11B, 11C - Graphs of Precision vs Recall tests of the Max, Min, Avg Poolings respectively

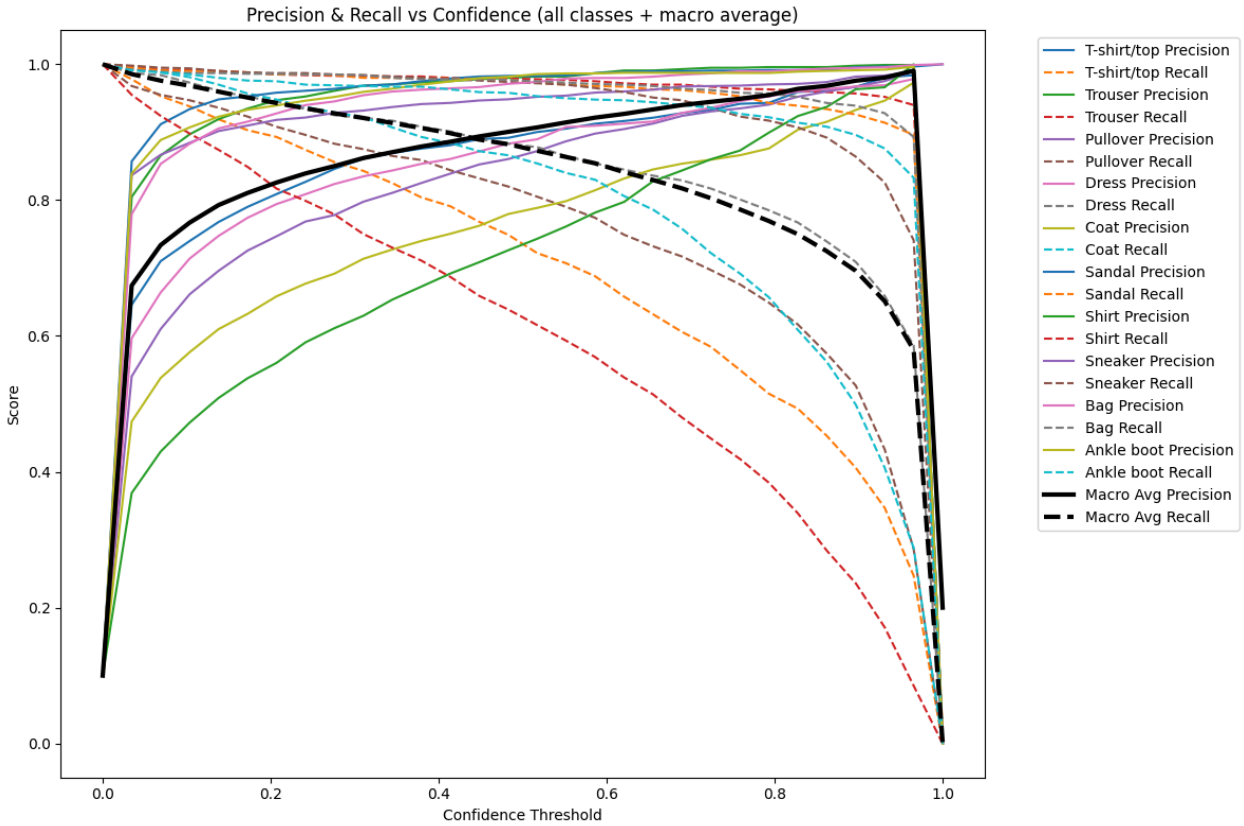
(Made by Myself - 2025)

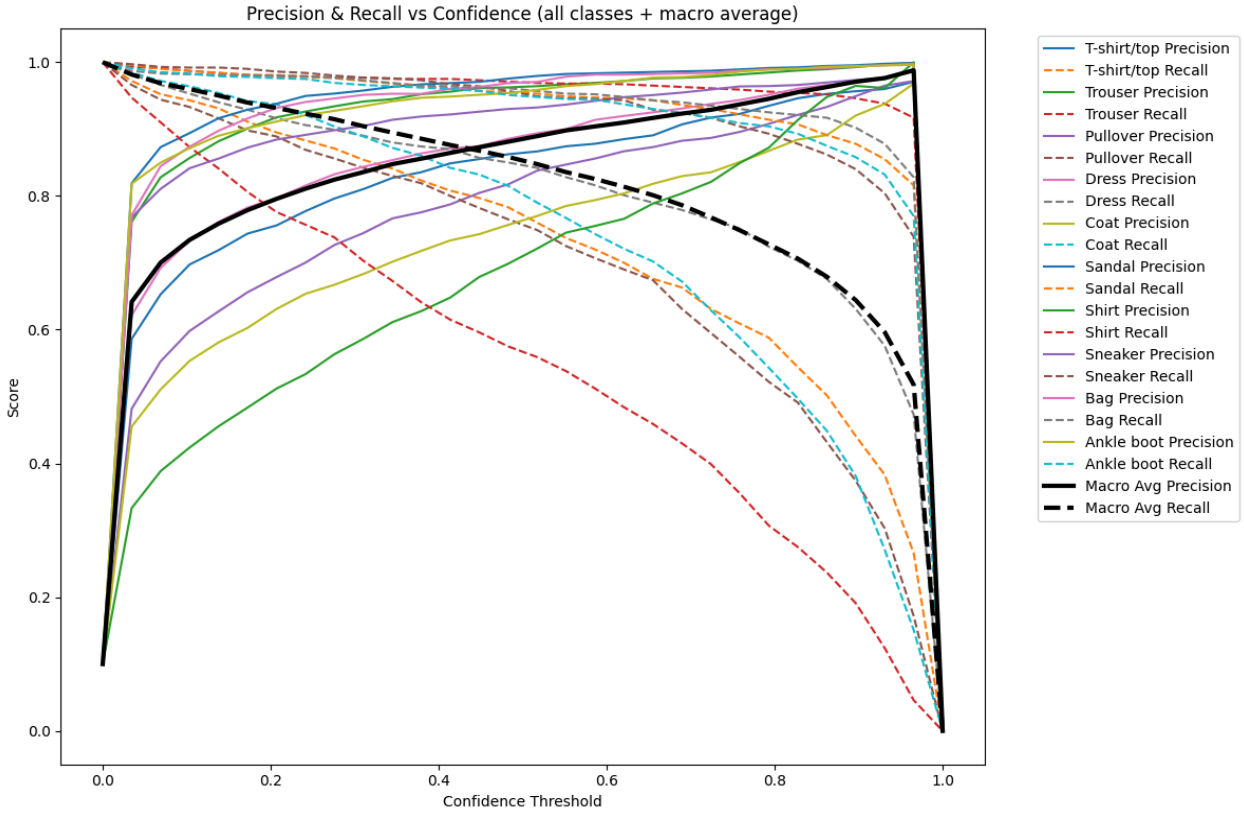
Precision-Recall graphs visualize how well a model can find the correct items and how many mistakes it makes while doing that. High recall but low precision symbolizes that the model finds many correct clothing types, but also makes a lot of mistakes. High precision but low recall means that the model is very careful and usually correct, but it misses many items.

By analysing the graphs, we can see that in the beginning the lines run smoothly and stay high, meaning that the models keep the balance between accuracy and recognition quality really well, but as we approach higher recall, lines sharply drop, symbolizing that the model is greatly sacrificing accuracy in order to find more objects.

Max pooling showed the highest results for this test, highlighting the performance of a model for sandals, bags and trousers, as their graphs stayed higher than others for higher recall score.

4.2.3 Precision & Recall on Confidence


Precision & Recall vs Confidence (all classes + macro average)
Score
Confidence Threshold
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top Precision
T-shirt/top Recall
Trouser Precision
Trouser Recall
Pullover Precision
Pullover Recall
Dress Precision
Dress Recall
Coat Precision
Coat Recall
Sandal Precision
Sandal Recall
Shirt Precision
Shirt Recall
Sneaker Precision
Sneaker Recall
Bag Precision
Bag Recall
Ankle boot Precision
Ankle boot Recall
Macro Avg Precision
Macro Avg Recall


Precision & Recall vs Confidence (all classes + macro average)
Score
Confidence Threshold
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top Precision
T-shirt/top Recall
Trouser Precision
Trouser Recall
Pullover Precision
Pullover Recall
Dress Precision
Dress Recall
Coat Precision
Coat Recall
Sandal Precision
Sandal Recall
Shirt Precision
Shirt Recall
Sneaker Precision
Sneaker Recall
Bag Precision
Bag Recall
Ankle boot Precision
Ankle boot Recall
Macro Avg Precision
Macro Avg Recall

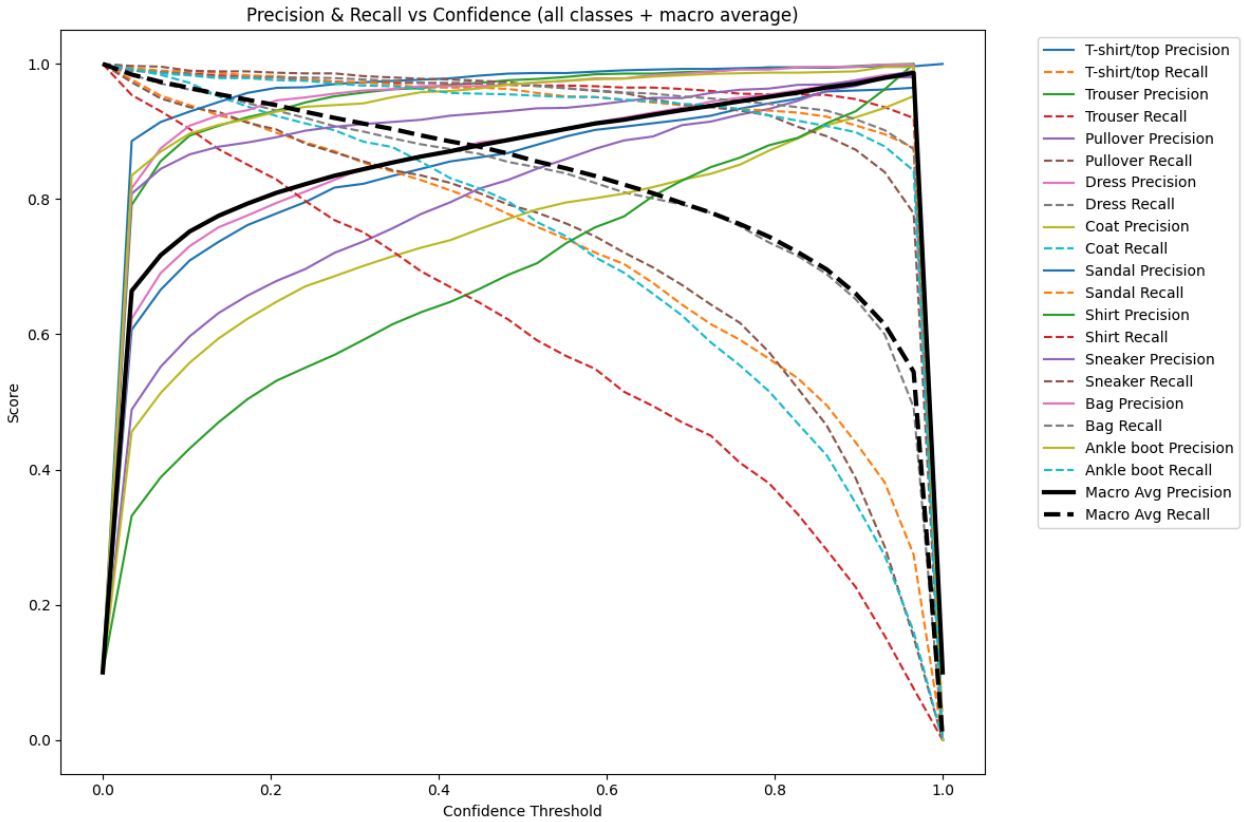


Fig. 12A, 12B, 12C - Graphs of Precision & Recall vs Confidence Threshold tests of the Max, Min, Avg Poolings respectively

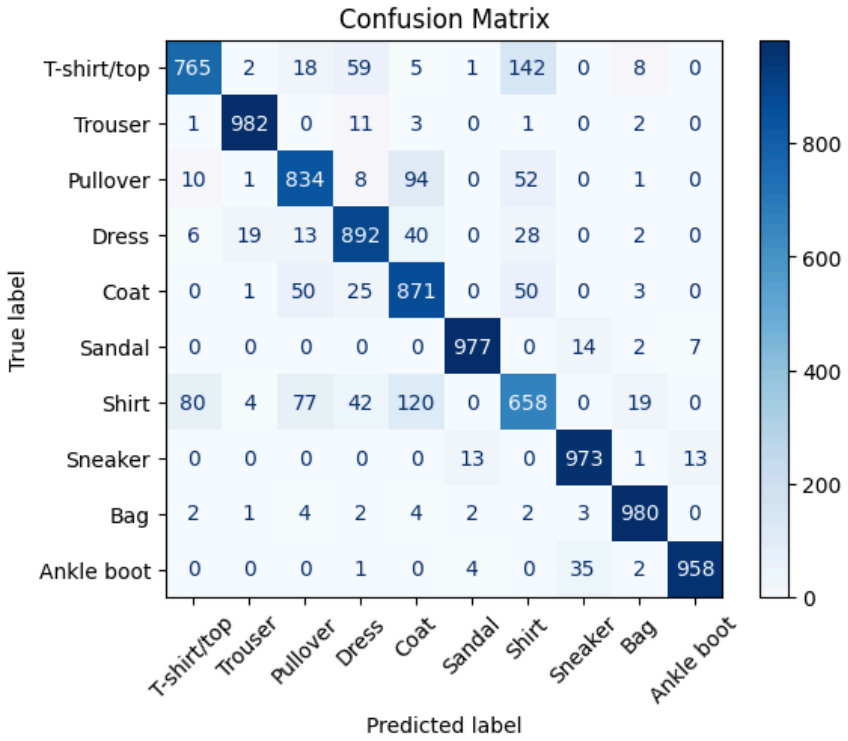
(Made by Myself - 2025)

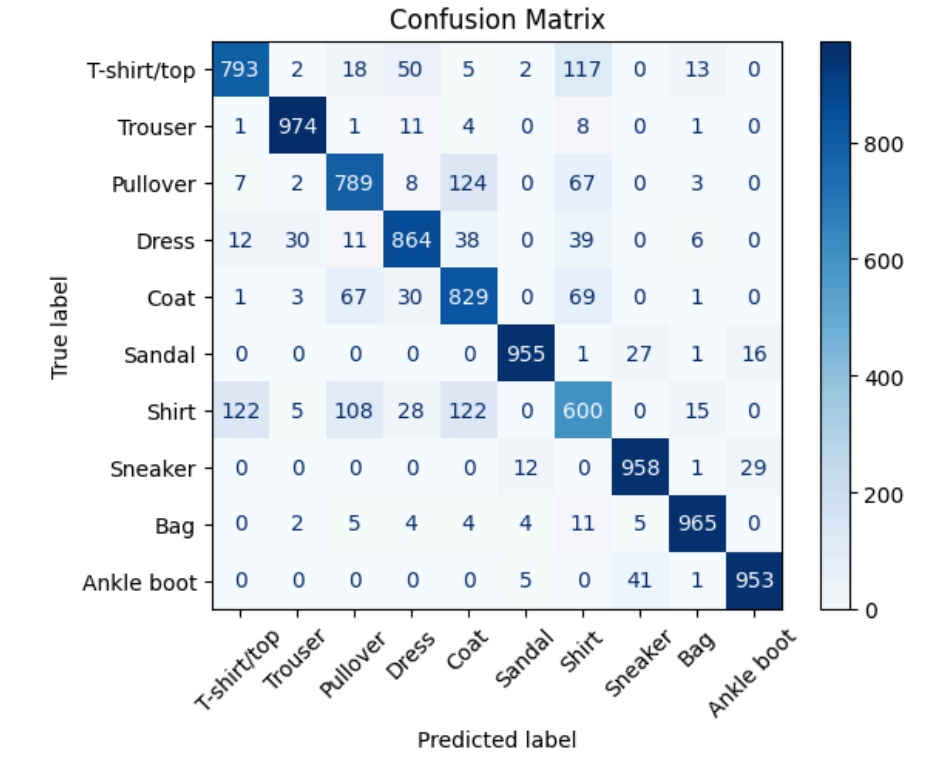
The graph shows how the Precision and Recall indicators change with an increase in the confidence threshold. At low threshold values, the model outputs almost all objects as positive, which gives a high Recall value (the model finds almost all true examples), but Precision is low, since there are many false positives. As the threshold increases, precision also increases, as only the most confident predictions remain, but Recall decreases and the model begins to skip more true examples.

The graph shows that almost all classes repeat this pattern, although the Precision and Recall varies for categories of clothing. For example, the "Router" and "Ankle boot" classes demonstrate high stability and achieve an almost perfect balance at all threshold values. While

"Shirt" and "Sandal" have a stronger gap between Precision and Recall, which indicates the difficulties of the model in classifying them.

4.2.4 Confusion Matrix




Confusion Matrix
T-shirt/top 793 2 18 50 5 2 117 0 13 0
Trouser 1 974 1 11 4 0 8 0 1 0
Pullover 7 2 789 8 124 0 67 0 3 0
Dress 12 30 11 864 38 0 39 0 6 0
Coat 1 3 67 30 829 0 69 0 1 0
Sandal 0 0 0 0 0 955 1 27 1 16
Shirt 122 5 108 28 122 0 600 0 15 0
Sneaker 0 0 0 0 0 12 0 958 1 29
Bag 0 2 5 4 4 4 11 5 965 0
Ankle boot 0 0 0 0 0 5 0 41 1 953
True label
T-shirt/top Trouser Pullover Dress Coat Sandal Shirt Sneaker Bag Ankle boot
Predicted label
800 600 400 200 0

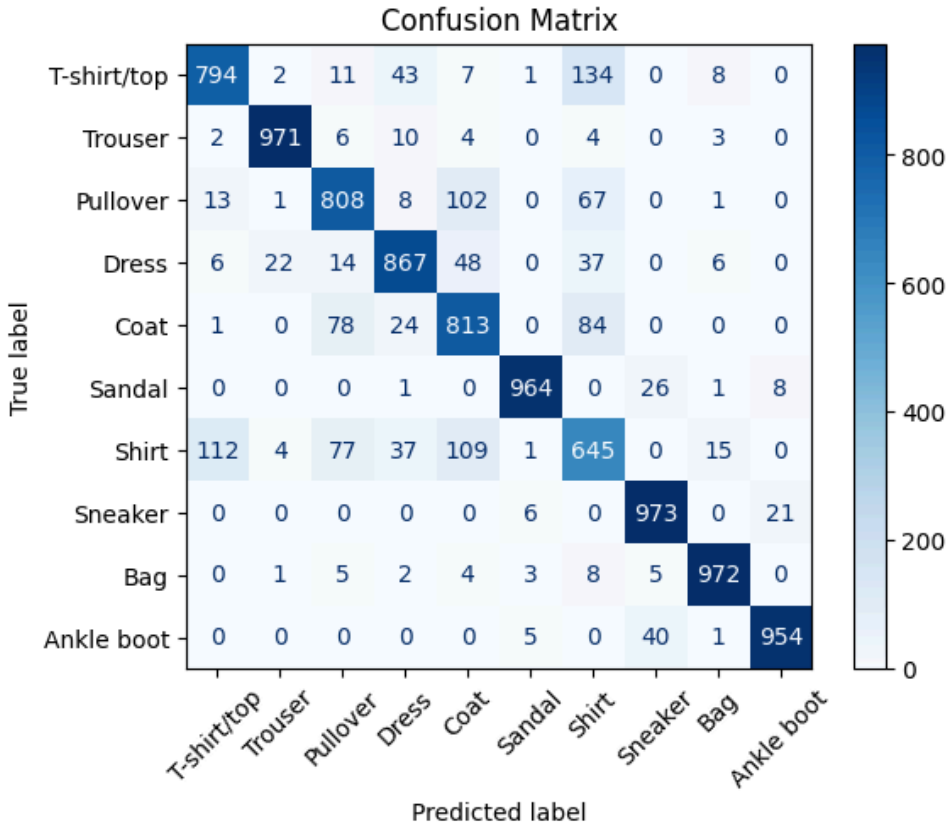

Confusion Matrix
T-shirt/top 794 2 11 43 7 1 134 0 8 0
Trouser 2 971 6 10 4 0 4 0 3 0
Pullover 13 1 808 8 102 0 67 0 1 0
Dress 6 22 14 867 48 0 37 0 6 0
Coat 1 0 78 24 813 0 84 0 0 0
Sandal 0 0 0 1 0 964 0 26 1 8
Shirt 112 4 77 37 109 1 645 0 15 0
Sneaker 0 0 0 0 0 6 0 973 0 21
Bag 0 1 5 2 4 3 8 5 972 0
Ankle boot 0 0 0 0 0 5 0 40 1 954
True label
T-shirt/top Trouser Pullover Dress Coat Sandal Shirt Sneaker Bag Ankle boot
Predicted label
800 600 400 200 0

Fig. 13A, 13B, 13C - Graphs of the Confusion Matrixes of the Max, Min, Avg Poolings respectively

(Made by Myself - 2025)

A confusion matrix shows how many items the model classified correctly, and which classes the model confuses with each other. By comparing the confusion matrices you can see how each pooling method affects mistakes across different classes.

Max pooling shows the strongest results, with most predictions lying on the diagonal. Classes like Trouser (982 correct), Sandal (977), Sneaker (973), and Bag (980) are recognized with almost no confusion. The main weakness is the Shirt class (658 correct), which is often mistaken for T-shirt/top (77), Pullover (42), and Coat (45).

Min pooling performs the worst. For example, Pullover drops to 789 correct, while Shirt falls to 600, with many errors against T-shirt/top (122) and Coat (108). Because Min pooling emphasizes the darkest pixels, it often loses important shape details and instead captures background noise, making it a weak choice for this dataset.

Average pooling stands between the two. It performs better than Min pooling but below Max pooling. It reduces the noise but also smooths out unique features, causing confusion in visually similar classes, though less severely than with Min pooling.

CONCLUSION

This investigation has explored and valued each pooling method to answer the given question *"How do different pooling strategies in convolutional neural network architectures (max, min, and average pooling) affect clothing recognition accuracy and feature extraction?"*. After conducting tests and evaluating the results, the best choice for clothing recognition is max pooling, as it showed the highest results for every given task.

This conclusion is supported by experiments like precision, recall, F1 score and confusion matrices, which were used to show the difference in results for three CNN models, each with a different pooling strategy.

Numerically, the max pooling model achieved the highest final test accuracy (0.889), lowest test loss (0.3027), highest average precision, recall and F1-score (0.8894, 0.889, 0.882, respectively) compared to average and min pooling, which scored lower.

This result can be explained by the fact of how max pooling works: by highlighting the strongest activations (edges and prominent features) it keeps contours and local features that are important for distinguishing shapes and textures. Average pooling smooths details and min pooling emphasized darker/background pixels, which reduced power for many classes in this dataset.

For future works on clothing recognition with similar data, max pooling is recommended. Future research should use colored datasets, vary kernel and stride, and test other alternatives (e.g. adaptive pooling or global pooling). For colored datasets I would suggest converting the images into black and white ones and then work with it. Afterwards you could get the initial (colored) images and assign the results to them.

In the end, pooling choice is only one of many architectural decisions, and this investigation shows that max pooling consistently improves clothing classification performance.

LIMITATIONS AND CHALLENGES

Since Max Pooling has shown the best results, I will start with discussing its own limitations, which surely impacted my exploration. Since max pooling keeps only one value per region, features like smooth gradients or light textures are usually being lost, which could blur or misrepresent object boundaries, and therefore cause similar objects to be determined wrong.

If we do research outside of this dataset, in real world situations there are a lot of disturbing bias moments that might affect productivity of each pooling method. Even though the Fashion-MNIST is used for benchmarking, it is still a simplified, grayscale, and pixelized dataset

that does not explore the full diversity of real clothing images. If certain clothing are over-represented, or shown from similar angles, the model can learn biased feature representations, for example: different lighting conditions, shifts in the angles, or coloured photos can completely lower the results. Different lightning conditions could make the contours less visible and therefore reduce the contrast that pooling layers rely on to find strong and weak activations. In these cases, max pooling may misclassify the classes because some edges become unnoticeable, while min pooling could see shadows as important features.

Moreover, variations in lighting, image quality, or camera angle can change how activations are found, leading to inconsistent pooling outputs. This means that a model trained under one lighting setup may perform poorly when applied to real-world data with uneven brightness or colour variation.

Training for only three epochs also limits the network's ability to fully converge. Accuracy was improving with each repetition, so more epochs could have improved the model's performance. Stopping early risks underfitting, meaning the model has not yet learned the full range of features in the dataset.

Another limitation is that Fashion-MNIST has 60,000 training and 10,000 testing images, which is decent but still relatively small compared to large-scale experiments. With more data, the results would be more accurate and confident.

BIBLIOGRAPHY

Source 1 :

Zafar, Afia, and Ali Arshad. *(Pdf) Convolutional Neural Networks: A Comprehensive Evaluation and Benchmarking of Pooling Layer Variants*, 16 Nov. 2024,

www.researchgate.net/publication/385750480_Convolutional_Neural_Networks_A_Comprehensive_Evaluation_and_Benchmarking_of_Pooling_Layer_Variants.

Source 2:

How to Backpropagate for CNNs, HogoNext Editorial team, 29 June 2025, hogonext.com/how-to-backpropagate-for-cnns/.

Source 3:

"Feature Map - Visualizing What a Convolutional Network Is Learning." - *Visualizing What a Convolutional Network Is Learning*, www.innovatiana.com/en/glossary/feature-map. Accessed 14 Nov. 2025.

Other Sources:

Agarwal, Nisha Bhadauriya, and Deepak Kumar Yadav. "An Overview of Different Object Detection Algorithms and Libraries." *An Overview of Different Object Detection Algorithms and Libraries*, 4 July 2024, www.researchgate.net/publication/382882639_An_Overview_of_Different_Object_Detection_Algorithms_and_Libraries.

Layton, Oliver. "CS343: Neural Networks Convolution and Max Pooling Oliver W. Layton." *Neural Networks, Convolution and Max Pooling*, Colby College, 2019, cs.colby.edu/courses/F19/cs343/lectures/lecture11/Lecture11Slides.pdf.

Rasul, Kashif. “Zalandoresearch/Fashion-Mnist: A Mnist-like Fashion Product Database. Benchmark.” *A MNIST-like Fashion Product Database.*, 22 July 2018, github.com/zalandoresearch/fashion-mnist.

Sharma, Pranshu. “Introduction to Convolutional Neural Networks in Deep Learning.” *Introduction to Convolutional Neural Networks in Deep Learning*, 19 Nov. 2024, www.analyticsvidhya.com/blog/2022/03/basic-introduction-to-convolutional-neural-network-in-deep-learning/.

APPENDICES

Appendix A: Investigation Code

(This appendix is made of images of my code to provide a clear presentation of my results and steps taken.)

[51] ✓ 0s

```
import torch
from torch import nn
import torchvision
from torchvision import datasets, transforms
from torchvision.transforms import ToTensor
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import DataLoader
```

[52] ✓ 0s

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

```
'cpu'
```

[53] ✓ 0s

```
def accuracy_fn(y_true, y_pred):
    correct = torch.eq(y_true, y_pred).sum().item()
    acc = (correct / len(y_pred)) * 100
    return acc
```

[54] ✓ 0s

```
train_data = datasets.FashionMNIST (
    root = 'data_2',
    train = True,
    target_transform= None,
    transform = torchvision.transforms.ToTensor(),
    download = True
)
test_data = datasets.FashionMNIST (
    root = 'data_2',
    train = False,
    target_transform= None,
    transform = torchvision.transforms.ToTensor(),
    download = True
)

class_names = train_data.classes
class_names
```

```
['T-shirt/top',
 'Trouser',
 'Pullover',
 'Dress',
 'Coat',
 'Sandal',
 'Shirt',
 'Sneaker',
 'Bag',
 'Ankle boot']
```

[55] ✓ 0s

```
BATCH_SIZE = 32

train_batch = DataLoader( shuffle=True,
                          dataset=train_data,
                          batch_size = BATCH_SIZE,
                          )

test_batch = DataLoader( shuffle=False,
                          dataset=test_data,
                          batch_size = BATCH_SIZE,
                          )

first_batch_imgs, first_batch_class = next(iter(train_batch))
first_batch_class

class_to_idx = train_data.class_to_idx
class_to_idx
```

```
{'T-shirt/top': 0,
 'Trouser': 1,
 'Pullover': 2,
 'Dress': 3,
 'Coat': 4,
 'Sandal': 5,
 'Shirt': 6,
 'Sneaker': 7,
 'Bag': 8,
 'Ankle boot': 9}
```

[56] ✓ 0s

```
# Custom MinPool2d
class MinPool2d(nn.Module):
    def __init__(self, kernel_size, stride=None, padding=0):
        super(MinPool2d, self).__init__()
        self.pool = nn.MaxPool2d(kernel_size, stride, padding)

    def forward(self, x):
        return -self.pool(-x)
```

[57]

```
class FashionMNISTModel(nn.Module):
  def __init__(self, input_layers, output_layers, hidden_units):
    super().__init__()
    self.stack_1 = nn.Sequential(
        nn.Conv2d(
            in_channels = input_layers,
            out_channels = hidden_units,
            kernel_size=3,
            stride = 1,
            padding = 1
        ),
        nn.ReLU(),
        nn.Conv2d(
            in_channels = hidden_units,
            out_channels = hidden_units,
            kernel_size=3,
            stride = 1,
            padding = 1
        ),
        nn.ReLU(),
        nn.AvgPool2d(kernel_size=2) # Switch here between Max, Min, and Avg Poolings
        )
    self.stack_2 = nn.Sequential(
        nn.Conv2d(
            in_channels = hidden_units,
            out_channels = hidden_units,
            kernel_size=3,
            stride = 1,
            padding = 1
        ),
        nn.ReLU(),
        nn.Conv2d(
            in_channels = hidden_units,
            out_channels = hidden_units,
            kernel_size=3,
            stride = 1,
            padding = 1
        ),
        nn.ReLU(),
        nn.AvgPool2d(kernel_size=2) # Switch here between Max, Min, and Avg Poolings
        )
```

[57]
```
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(in_features=hidden_units*7*7,
                      out_features = output_layers)
          )

    def forward(self, x):
        x = self.stack_1(x)
        x = self.stack_2(x)
        x = self.classifier(x)
        return x
```

[58] ✓ 0s
```
torch.manual_seed(8)
model = FashionMNISTModel(input_layers=1,
                          hidden_units=10,
                          output_layers=len(class_names)).to(device)
```

[59] ✓ 0s
```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(params=model.parameters(), lr= 0.1)
```

[60] ✓ 0s
```
def train(model, data_loader, loss_fn, optimizer, accuracy_fn, device):
    train_acc, train_loss = 0, 0
    model.train()
    for batch, (X,y) in enumerate(data_loader):
        X,y = X.to(device), y.to(device)
        y_pred = model(X)
        loss = loss_fn(y_pred, y)
        train_loss += loss
        train_acc += accuracy_fn(y_true=y, y_pred=y_pred.argmax(dim=1))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    train_loss /= len(data_loader)
    train_acc /= len(data_loader)
    print(f'train_loss: {train_loss:.5f} | train_accuracy: {train_acc:.2f}')

def test(model, data_loader, loss_fn, accuracy_fn, device):
    test_loss, test_acc = 0,0
    model.eval()
    with torch.inference_mode():
        for X,y in data_loader:
            X,y = X.to(device), y.to(device)
            test_pred = model(X)
            test_loss += loss_fn(test_pred, y)
            test_acc += accuracy_fn(y_true=y, y_pred = test_pred.argmax(dim=1))

        test_loss /= len(data_loader)
        test_acc /= len(data_loader)
        print(f'test_loss: {test_loss:.5f}| test_acc:{test_acc:.2f}')
```

[61] ✓ 0s

```python
def eval(model, data_loader, loss_fn, accuracy_fn, device=device):
  loss, acc =0,0
  model.eval()
  with torch.inference_mode():
    for X,y in data_loader:
       X.to(device), y.to(device)
       y_pred = model(X)
       loss += loss_fn(y_pred, y)
       acc += accuracy_fn(y_true=y, y_pred=y_pred.argmax(dim=1))

    loss /= len(data_loader)
    acc /= len(data_loader)
    return {"model_name": model.__class__.__name__,
          'model_loss': loss.item(),
          'model_acc': acc}
```

+ Code + Text

[62]

```python
epochs = 3
for epoch in range(epochs):
  train(model=model, data_loader=train_batch, loss_fn= loss_fn,
        accuracy_fn=accuracy_fn, optimizer=optimizer, device=device)
  test(model=model, data_loader=test_batch, loss_fn= loss_fn,
       accuracy_fn=accuracy_fn, device=device)
```

```
train_loss: 0.66987 | train_accuracy: 75.36
test_loss: 0.41413| test_acc:85.20
train_loss: 0.37740 | train_accuracy: 86.24
test_loss: 0.37582| test_acc:86.31
train_loss: 0.33194 | train_accuracy: 87.94
test_loss: 0.33363| test_acc:87.62
```

[63] ✓ 5s

```python
#Calculate model_0 results on test dataset
model_results = eval(model=model,
                     data_loader=test_batch,
                     loss_fn=loss_fn,
                     accuracy_fn=accuracy_fn,
                     device=device)
model_results
```

```
{'model_name': 'FashionMNISTModel',
 'model_loss': 0.3336285352706909,
 'model_acc': 87.61980830670926}
```

[64] ✓ 1s

```python
import random

random.seed(50)
test_samples = []
test_labels = []
for sample, label in random.sample(list(test_data), k=9):
  test_samples.append(sample)
  test_labels.append(label)

test_samples[0].shape
```

```
torch.Size([1, 28, 28])
```

[65]
✓ 0s

```
def make_predictions(model: torch.nn.Module,
                     data: list,
                     device: torch.device = device):
  pred_probs = []
  model.eval()
  model.to(device)
  with torch.inference_mode():
    for sample in data:
      #Prepare the sample(add a batch dimension and pass to target device)
      sample = torch.unsqueeze(sample, dim=0).to(device)

      #Forward pass (model outputs raw logits)
      pred_logit = model(sample)

      #Get prediction probability (logit -> prediction probability)
      pred_prob = torch.softmax(pred_logit.squeeze(), dim=0)

      #get pred_prob off the GPU for further calc
      pred_probs.append(pred_prob.cpu())

  #Stack the pred_probs to turn list into a tensor
  return torch.stack(pred_probs)

#make predictions
pred_probs = make_predictions(model=model,
                              data=test_samples)

#Convert pred probabilities to labels
pred_classes = pred_probs.argmax(dim=1)
```

[66]
✓ 0s

```
#Plot preds
plt.figure(figsize=(9,9))
nrows = 3
ncols = 3
for i, sample in enumerate(test_samples):
  #Create subplot
  plt.subplot(nrows, ncols, i+1)
  plt.imshow(sample.squeeze(), cmap='gray')

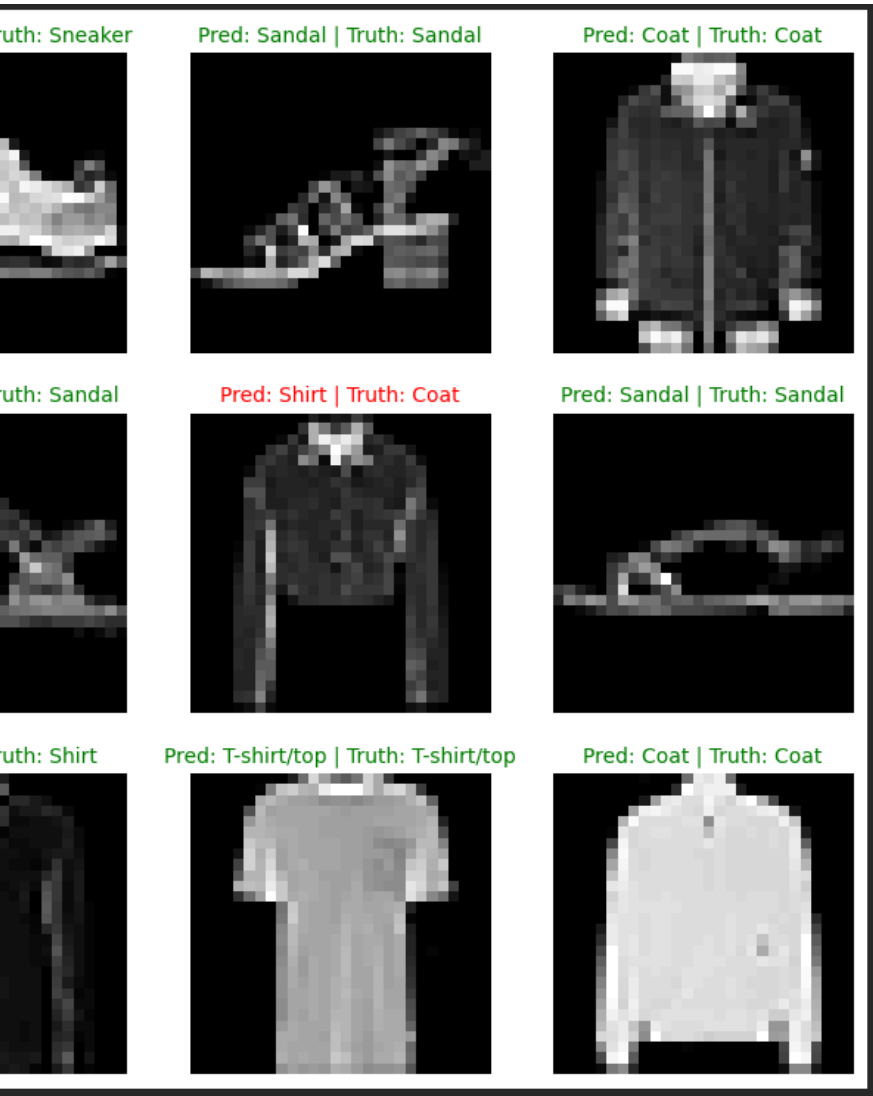
  # Find the prediction (in text form, e.q 'sandal')
  pred_label = class_names[pred_classes[i]]

  #Get the truth label (in text form)
  truth_label = class_names[test_labels[i]]

  #Create title
  title_text = f'Pred: {pred_label} | Truth: {truth_label}'

  if pred_label == truth_label:
    plt.title(title_text, fontsize=10, c='g')
  else:
    plt.title(title_text, fontsize=10, c='r')

  plt.axis(False)
```



[67]
0s

```
from sklearn.metrics import precision_score, recall_score, f1_score, classification_report
```

[68]

```python
def precision_recall_f1(model, data_loader, device=device):
    model.eval()
    y_true = []
    y_pred = []

    with torch.inference_mode():
        for X, y in data_loader:
            X, y = X.to(device), y.to(device)
            preds = model(X).argmax(dim=1)

            y_true.extend(y.cpu().numpy())
            y_pred.extend(preds.cpu().numpy())

    # Weighted = accounts for class imbalance, Macro = equal weight for each class
    precision = precision_score(y_true, y_pred, average="weighted")
    recall = recall_score(y_true, y_pred, average="weighted")
    f1 = f1_score(y_true, y_pred, average="weighted")

    print("Precision:", precision)
    print("Recall:", recall)
    print("F1-score:", f1)
    print("\nClassification report:\n", classification_report(y_true, y_pred,
                                                             target_names=class_names))

    return precision, recall, f1
```

[69] ✓ 4s

```python
precision, recall, f1 = precision_recall_f1(model, test_batch, device=device)
```

```
Precision: 0.8766607369404253
Recall: 0.8761
F1-score: 0.8760586010258048

Classification report:
              precision    recall  f1-score   support

 T-shirt/top       0.86      0.79      0.82      1000
     Trouser       0.97      0.97      0.97      1000
    Pullover       0.81      0.81      0.81      1000
       Dress       0.87      0.87      0.87      1000
        Coat       0.75      0.81      0.78      1000
      Sandal       0.98      0.96      0.97      1000
       Shirt       0.66      0.65      0.65      1000
     Sneaker       0.93      0.97      0.95      1000
         Bag       0.97      0.97      0.97      1000
  Ankle boot       0.97      0.95      0.96      1000

    accuracy                           0.88     10000
   macro avg       0.88      0.88      0.88     10000
weighted avg       0.88      0.88      0.88     10000
```

[70] ✓ 0s

```python
from sklearn.metrics import precision_recall_curve, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import numpy as np

def get_all_preds(model, data_loader, device=device):
    model.eval()
    y_true, y_pred, y_probs = [], [], []

    with torch.inference_mode():
        for X, y in data_loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            probs = torch.softmax(logits, dim=1)   # class probabilities
            preds = probs.argmax(dim=1)

            y_true.extend(y.cpu().numpy())
            y_pred.extend(preds.cpu().numpy())
            y_probs.extend(probs.cpu().numpy())

    return np.array(y_true), np.array(y_pred), np.array(y_probs)
```

[71] ✓ 4s

```python
y_true, y_pred, y_probs = get_all_preds(model, test_batch, device)
```

[72] ✓ 0s

```python
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt
import numpy as np

def plot_pr_curves(y_true, y_probs, class_names):
    plt.figure(figsize=(15, 10))

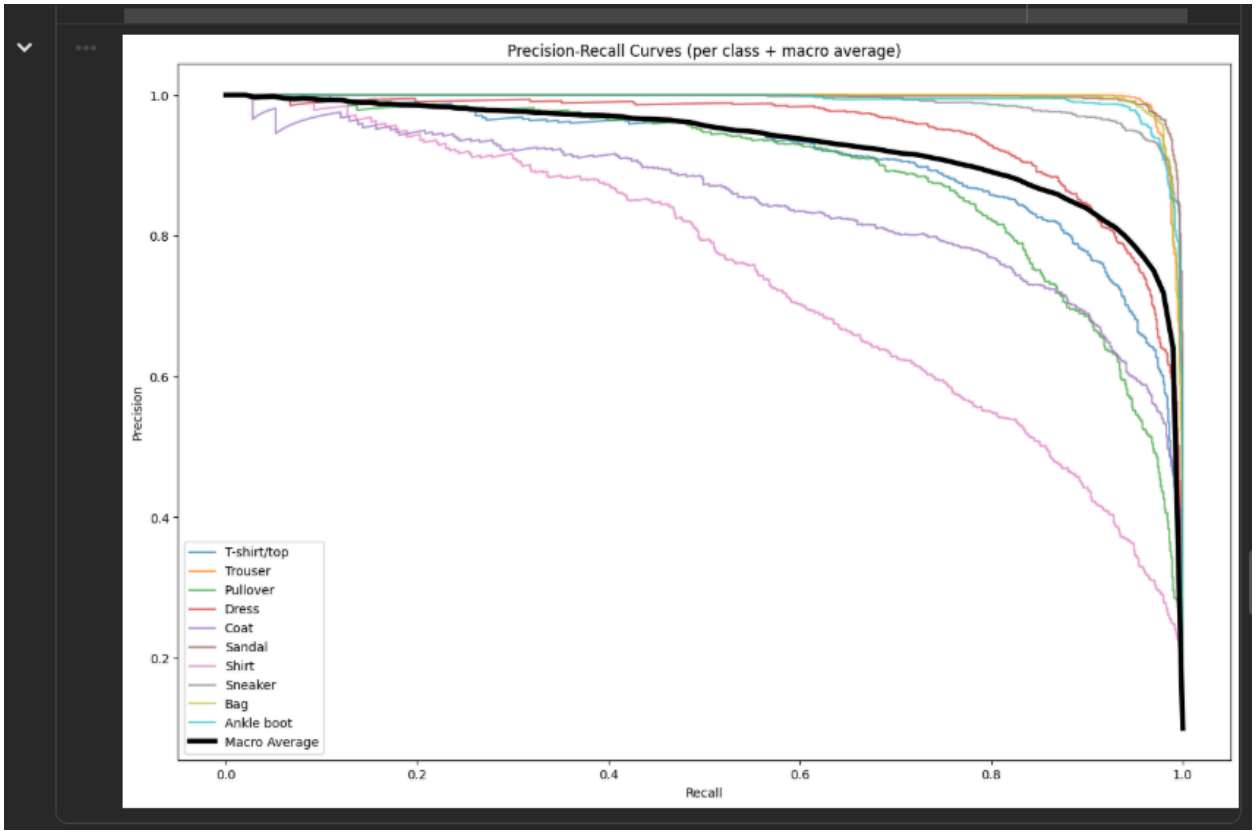
    # --- One-vs-Rest curves per class ---
    precisions_list, recalls_list = [], []
    for i, name in enumerate(class_names):
        precision, recall, _ = precision_recall_curve(
            (y_true == i).astype(int), y_probs[:, i]
        )
        plt.plot(recall, precision, label=name, alpha=0.7)
        precisions_list.append(np.interp(np.linspace(0,1,100), recall[::-1], precision[::-1]))
        recalls_list.append(np.linspace(0,1,100))

    # --- Macro-average curve ---
    precisions_macro = np.mean(precisions_list, axis=0)
    recalls_macro = np.linspace(0,1,100)

    plt.plot(
        recalls_macro,
        precisions_macro,
        label="Macro Average",
        linewidth=4,
        color="black"
    )

    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title("Precision-Recall Curves (per class + macro average)")
    plt.legend()
    plt.show()


plot_pr_curves(y_true, y_probs, class_names)
```


Precision-Recall Curves (per class + macro average)
Precision
Recall
1.0
0.8
0.6
0.4
0.2
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle boot
Macro Average

[73]

```python
from sklearn.metrics import f1_score
import numpy as np
import matplotlib.pyplot as plt

def plot_f1_vs_confidence(y_true, y_probs, class_names):
    thresholds = np.linspace(0, 1, 30)
    plt.figure(figsize=(15, 10))

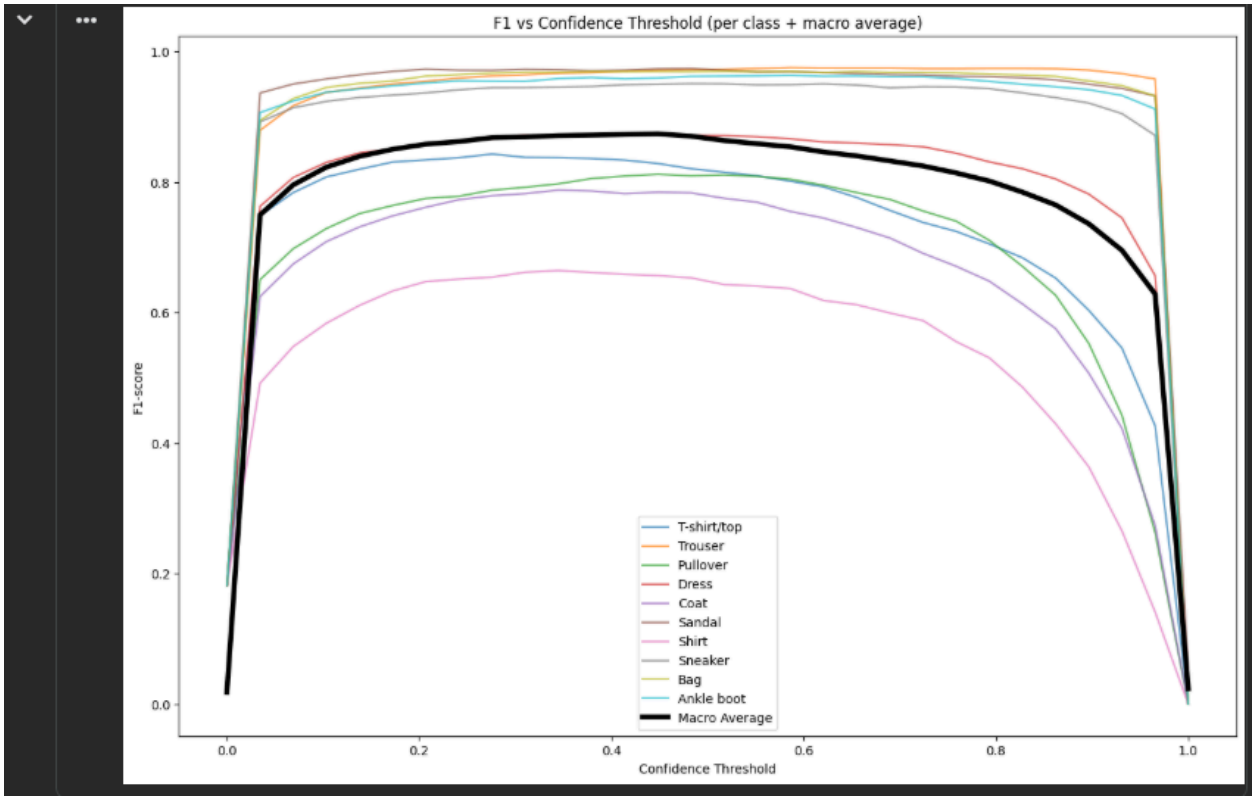
    # --- Per-class curves ---
    for i, name in enumerate(class_names):
        f1_scores = []
        for t in thresholds:
            preds_t = (y_probs[:, i] >= t).astype(int)
            f1_scores.append(
                f1_score((y_true == i).astype(int), preds_t, zero_division=0)
            )
        plt.plot(thresholds, f1_scores, label=name, alpha=0.7)

    # --- Macro-average curve ---
    f1_macro = []
    for t in thresholds:
        preds_t = (y_probs >= t).astype(int)
        f1_macro.append(
            f1_score(y_true, preds_t.argmax(axis=1), average="macro", zero_division=0)
        )

    plt.plot(
        thresholds,
        f1_macro,
        label="Macro Average",
        linewidth=4,
        color="black"
    )

    plt.xlabel("Confidence Threshold")
    plt.ylabel("F1-score")
    plt.title("F1 vs Confidence Threshold (per class + macro average)")
    plt.legend()
    plt.show()

plot_f1_vs_confidence(y_true, y_probs, class_names)
```


F1 vs Confidence Threshold (per class + macro average)
F1-score
Confidence Threshold
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle boot
Macro Average

[74]

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_score, recall_score

def plot_pr_vs_confidence_all(y_true, y_probs, class_names, n_thresholds=30):
    thresholds = np.linspace(0, 1, n_thresholds)

    plt.figure(figsize=(12, 8))

    n_classes = y_probs.shape[1]

    macro_precisions = []
    macro_recalls = []

    for t in thresholds:
        per_class_prec = []
        per_class_rec = []
        for i in range(n_classes):
            y_true_i = (y_true == i).astype(int)
            preds_i = (y_probs[:, i] >= t).astype(int)
            per_class_prec.append(precision_score(y_true_i, preds_i, zero_division=0))
            per_class_rec.append(recall_score(y_true_i, preds_i, zero_division=0))
        macro_precisions.append(np.mean(per_class_prec))
        macro_recalls.append(np.mean(per_class_rec))

    for i, name in enumerate(class_names):
        precisions, recalls = [], []
        for t in thresholds:
            preds_t = (y_probs[:, i] >= t).astype(int)
            precisions.append(precision_score((y_true == i).astype(int),
                                              preds_t, zero_division=0))
            recalls.append(recall_score((y_true == i).astype(int),
                                        preds_t, zero_division=0))

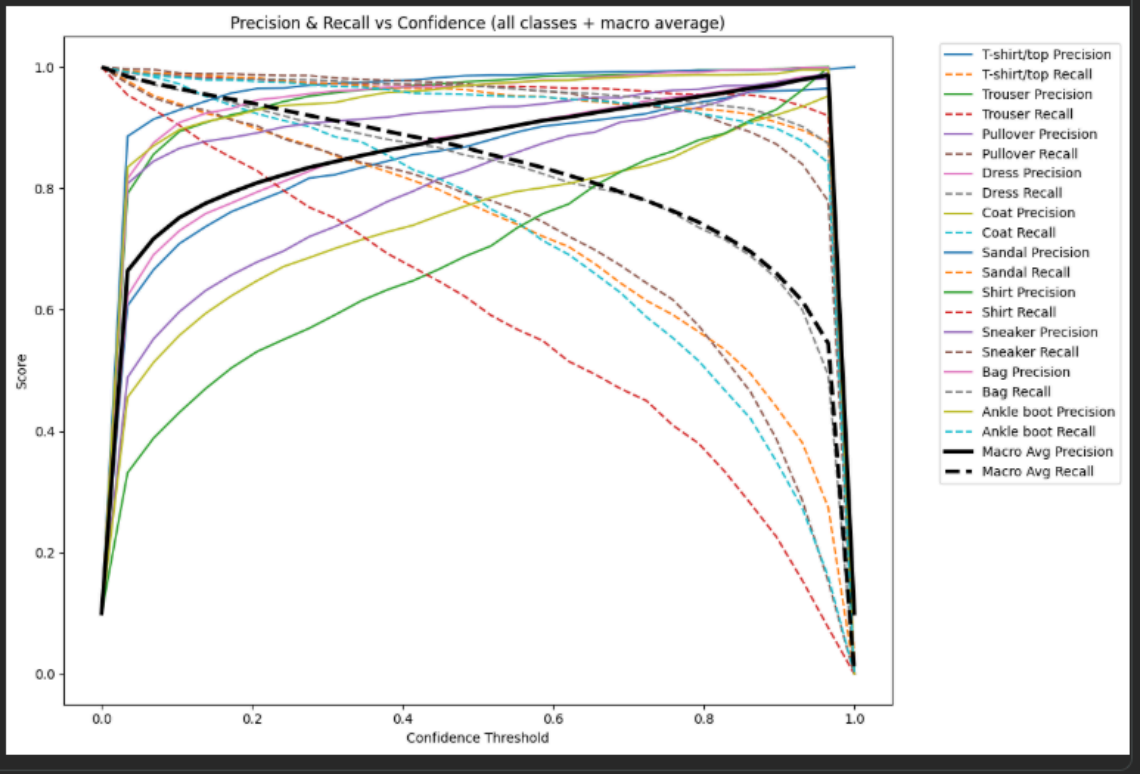
        plt.plot(thresholds, precisions, label=f"{name} Precision")
        plt.plot(thresholds, recalls, linestyle="--", label=f"{name} Recall")

    plt.plot(thresholds, macro_precisions, color="black", linewidth=3,
             label="Macro Avg Precision")
    plt.plot(thresholds, macro_recalls, color="black", linewidth=3, linestyle="--",
             label="Macro Avg Recall")
```

[74]

```
    plt.xlabel("Confidence Threshold")
    plt.ylabel("Score")
    plt.title("Precision & Recall vs Confidence (all classes + macro average)")
    plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
    plt.tight_layout()
    plt.show()


plot_pr_vs_confidence_all(y_true, y_probs, class_names)
```



[75]
✓ 0s

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(cmap="Blues", xticks_rotation=45, values_format="d")
plt.title("Confusion Matrix")
plt.show()
```

